

Date: 29-12-2025 à Day 2 – Querying & Modifying Data

By Shaaz

1. Create Database command.

```
CREATE DATABASE InsuranceDB
```

2. Create table commands for all the tables with constraints, relationships etc.

```
CREATE TABLE Customers (  
    CustomerID INT IDENTITY PRIMARY KEY,  
    FirstName VARCHAR(20) NOT NULL,  
    LastName VARCHAR(20) ,  
    DateOfBirth DATE NOT NULL,  
    Phone VARCHAR(15) NOT NULL,  
    Email VARCHAR(50) NOT NULL UNIQUE  
);
```

```
CREATE TABLE Agents (  
    AgentId INT IDENTITY PRIMARY KEY,  
    AgentName VARCHAR(20) NOT NULL,  
    Phone VARCHAR(15) NOT NULL,  
    City VARCHAR(20) NOT NULL  
);
```

```
CREATE TABLE Policies (  
    PolicyId INT IDENTITY PRIMARY KEY,  
    PolicyName VARCHAR(20) NOT NULL,  
    PolicyType VARCHAR(20) NOT NULL,  
    PremiumAmount MONEY NOT NULL,  
    DurationYears INT NOT NULL  
);
```

```
CREATE TABLE PolicyAssignments (  
    AssignmentID INT IDENTITY PRIMARY KEY,  
    CustomerID INT NOT NULL,  
    PolicyId INT NOT NULL,
```

```

AgentId INT NOT NULL,
StartDate DATE NOT NULL,
EndDate DATE NOT NULL,

CONSTRAINT FK_PA_Customers
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID),

CONSTRAINT FK_PA_Policies
    FOREIGN KEY (PolicyId) REFERENCES Policies(PolicyId),

CONSTRAINT FK_PA_Agents
    FOREIGN KEY (AgentId) REFERENCES Agents(AgentId)
);

CREATE TABLE Claims (
    ClaimsID INT PRIMARY KEY,
    AssignmentID INT NOT NULL,
    ClaimDate DATE NOT NULL,
    ClaimAmount MONEY NOT NULL,
    ClaimStatus VARCHAR(20) NOT NULL,
    CONSTRAINT FK_Claims_Assignments
        FOREIGN KEY (AssignmentID) REFERENCES PolicyAssignments(AssignmentID),
);

```

3. Insert commands for all tables.

```

INSERT INTO Customers (FirstName, LastName, DateOfBirth, Phone, Email)
values ('Amit', 'Sharma', '1990-05-10', '9876543210', 'amit@gmail.com');
INSERT INTO Customers (FirstName, LastName, DateOfBirth, Phone, Email)
values ('Shaaz', 'Hussain', '1990-05-11', '9876543230', 'shaaz@gmail.com');

INSERT INTO Customers (FirstName, LastName, DateOfBirth, Phone, Email)
values ('Bharath', 'Simha', '1990-05-13', '9876543240', 'Bharath@gmail.com');

INSERT INTO Policies (PolicyName, PolicyType, PremiumAmount, DurationYears)
VALUES ('Health Secure', 'Health', 12000.00, 1)
INSERT INTO Policies (PolicyName, PolicyType, PremiumAmount, DurationYears)
VALUES ('Life Shield', 'Life', 25000.00, 1)

```

```
INSERT INTO Policies (PolicyName, PolicyType, PremiumAmount, DurationYears)
VALUES ('Motor Protect', 'Motor', 8000.00, 1)
```

4. Select commands

```
select * from Customers;
select * from Agents;
select * from Policies;
select * from PolicyAssignments;
select * from Claims;
```

SQLQuery_1 - (70) Local Docker SQL Server

Database: InsuranceDB

Results

CustomerID	FirstName	LastName	DateOfBirth	Phone	Email	Address	City
1	Amit	Sharma	1990-05-10	9876543210	amit@gmail.com	NULL	NULL
2	Shaaz	Hussain	1990-05-11	9876543230	shaaz@gmail.com	NULL	NULL
3	Bharath	Simha	1990-05-13	9876543240	Bharath@gmail.com	NULL	NULL
4	nani	gunji	2002-01-01	81828181	nani@gmail.com	NULL	NULL

AgentId	AgentName	Phone	City	Dev0fId
1	Ravi Kumar	9123456789	Hyderabad	NULL
2	Anita Verma	9234567890	Bangalore	NULL

PolicyId	PolicyName	PolicyType	PremiumAmount	DurationYears
1	Health Secure	Health	12000.00	1
2	Term Life	Term Life Shield	25000.00	1
3	Motor Protect	Motor	8000.00	1

AssignmentID	CustomerID	PolicyId	AgentId	StartDate	EndDate
1	3	1	1	2024-01-01	2024-12-31
2	4	2	2	2024-02-01	2025-01-31
3	5	3	1	2024-03-01	2025-02-28

Ln 72, Col 1 (125 selected) Spaces: 4 UTF-8 LF 15 rows MSSQL 00:00:00 127.0.0.1,1433 : InsuranceDB (70)

1. View all records Customers table.

```
select * from Customers;
```

2. View all records of PolicyAssignment table with CustomerId, PolicyId, StartDate and EndDate columns only.

```
SELECT CustomerID,PolicyId,StartDate,EndDate from PolicyAssignments;
```

The screenshot shows the Azure Data Studio interface. The main editor displays a SQL query with line numbers 192 to 208. The query is a multi-table join that includes a final SELECT statement. The 'Results' pane at the bottom shows a table with 3 rows and 4 columns: CustomerID, PolicyId, StartDate, and EndDate. The status bar at the bottom indicates the current cursor position and database context.

```
192 on c.CustomerID=p.CustomerID
193 join Claims c1
194 on c1.AssignmentID=p.AssignmentID
195 join Agents a
196 on p.AgentId=a.AgentId
197 join Policies p1
198 on p1.PolicyId=p.PolicyId
199 order by c1.ClaimAmount desc
200
201
202
203 -- Assignment
204
205 -- View all records of PolicyAssignment table with CustomerId, PolicyId, StartDate and
206 -- EndDate columns only.
207 SELECT CustomerID,PolicyId,StartDate,EndDate from PolicyAssignments;
208
```

	CustomerID	PolicyId	StartDate	EndDate
1	1	1	2024-01-01	2024-12-31
2	2	3	2024-02-01	2025-01-31
3	3	2	2024-03-01	2025-02-28

Ln 207, Col 1 (68 selected) Spaces: 4 UTF-8 LF 3 rows MSSQL 00:00:00 127.0.0.1,1433 : InsuranceDB (70)

3. Display all policies of Health type.

```
SELECT CustomerID,PolicyId,StartDate,EndDate from PolicyAssignments;
```

The screenshot shows the SQL Server Enterprise Manager interface. On the left, the 'SERVERS' tree is expanded to 'Local Docker SQL Server' > 'Databases' > 'InsuranceDB' > 'Tables'. The 'Sales' table is selected. The main pane shows a query editor with the following SQL query:

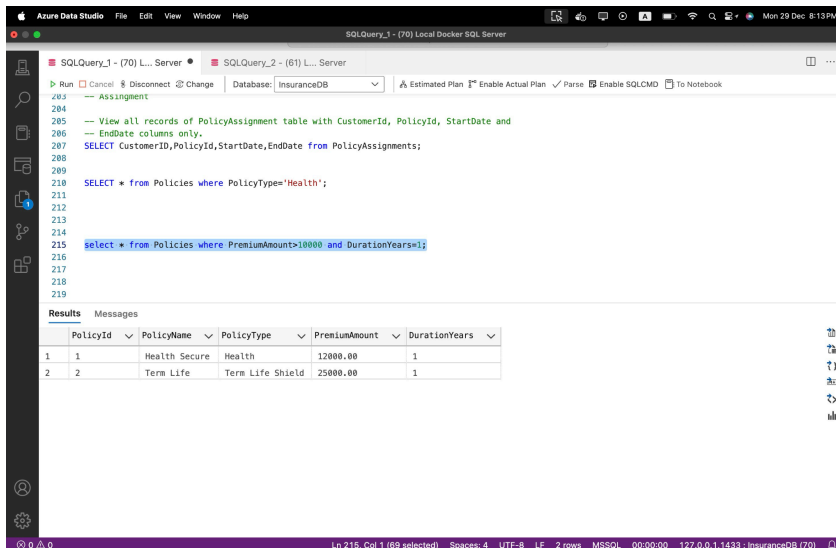
```
SELECT * from Policies where PolicyType='Health';
```

The query is executed, and the results are displayed in a table with the following columns: PolicyId, PolicyName, PolicyType, PremiumAmount, and DurationYears. The results table contains one row:

PolicyId	PolicyName	PolicyType	PremiumAmount	DurationYears
1	Health Secure	Health	12000.00	1

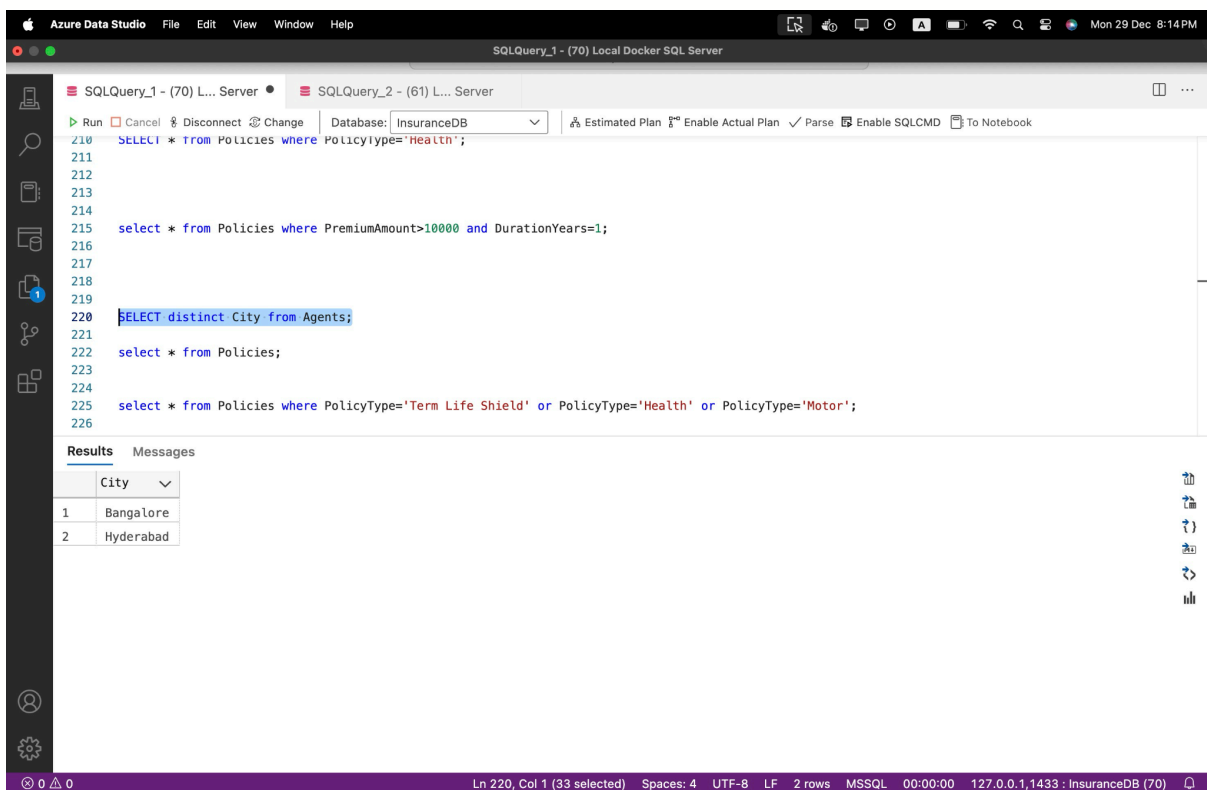
4. Display policies having premium amount more than 10000 and DurationYears is 1.

```
select * from Policies where PremiumAmount>10000 and  
DurationYears=1;
```



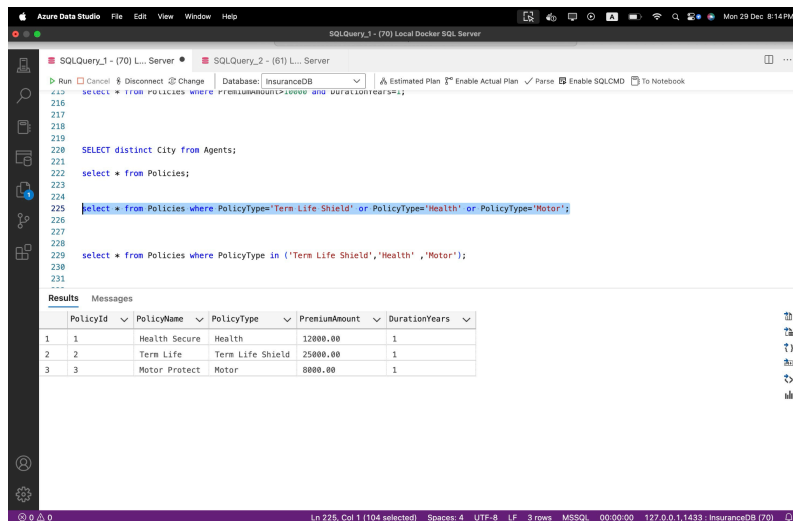
5. Display unique city names from where agents belong to.

```
SELECT distinct City from Agents;
```



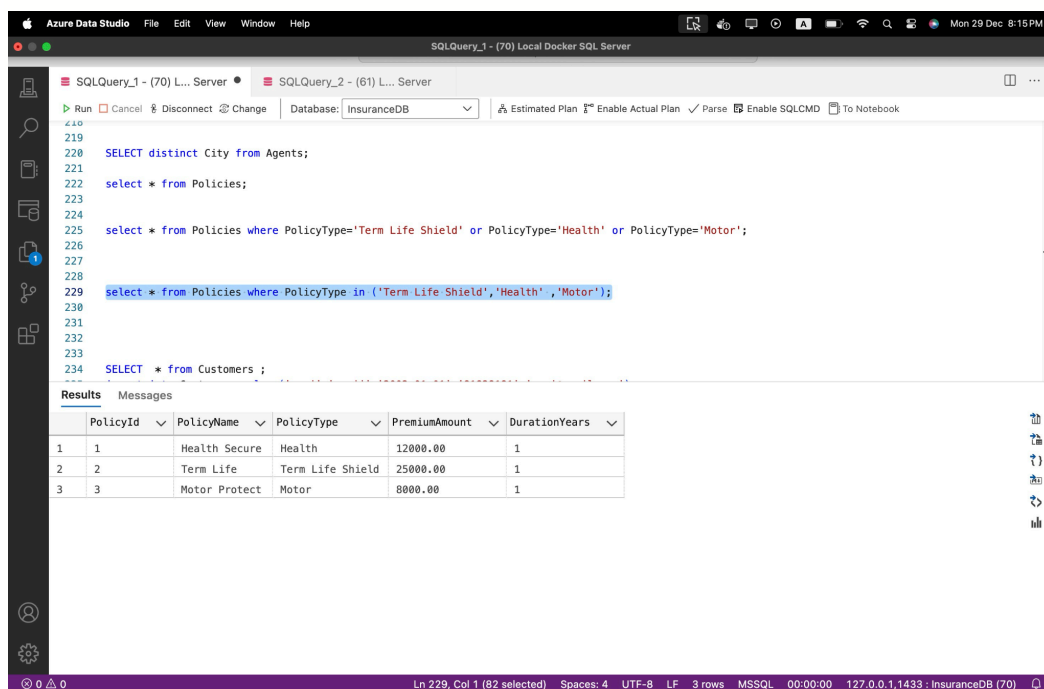
6. List policies of type Life, Health, Motor use OR clause.

```
select * from Policies where PolicyType='Term Life Shield'
or PolicyType='Health' or PolicyType='Motor';
```



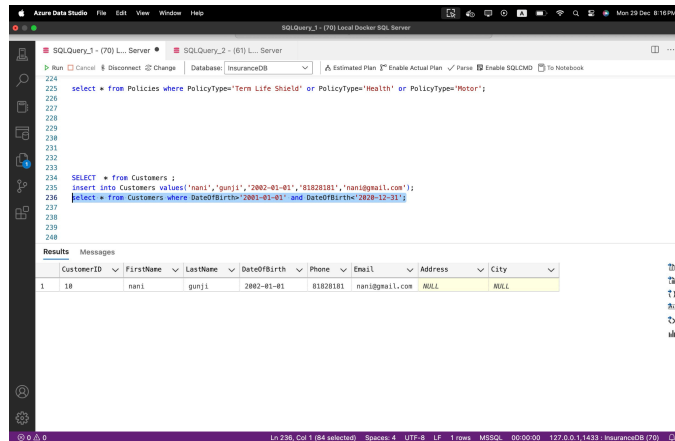
7. List policies of type Life, Health, Motor use IN operator.

```
select * from Policies where PolicyType in ('Term Life
Shield', 'Health' , 'Motor');
```



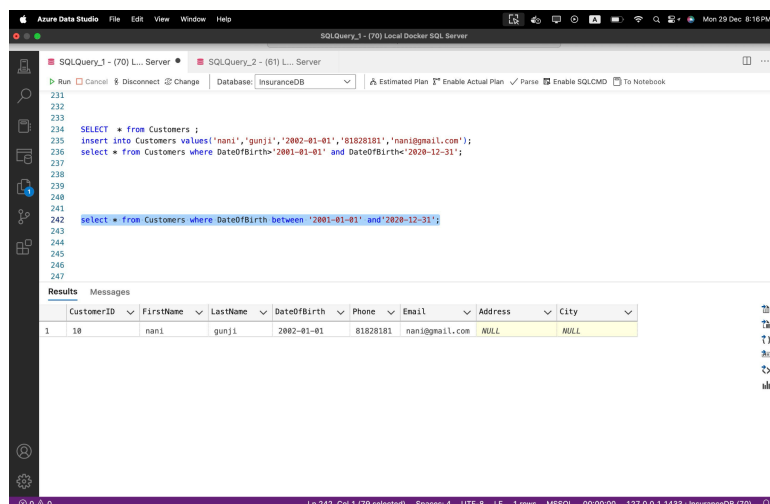
8. Display list of customers born after January 1 st , 2001 and before December 31 st , 2020 using >= and <= operators.

```
select * from Customers where DateOfBirth>'2001-01-01' and  
DateOfBirth<'2020-12-31';
```



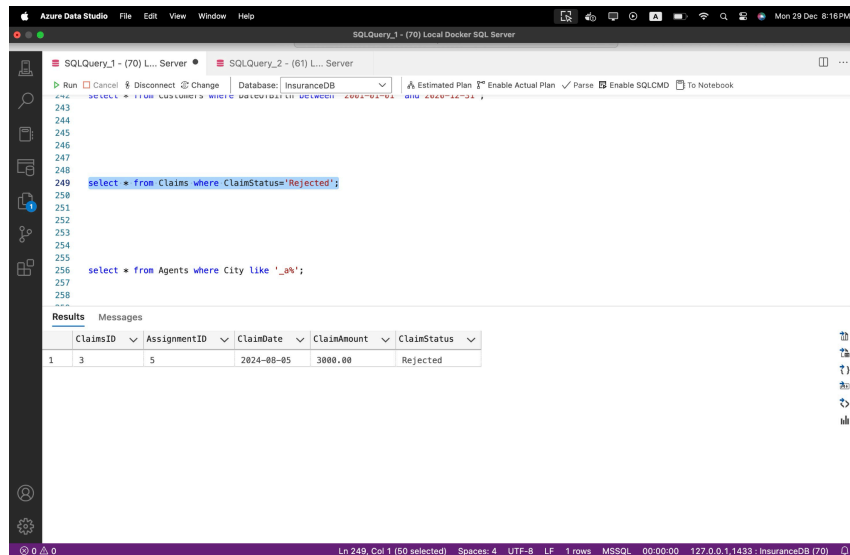
9. Display list of customers born after January 1 st , 2001 and before December 31 st , 2020 using between operator.

```
select * from Customers where DateOfBirth between  
'2001-01-01' and'2020-12-31';
```



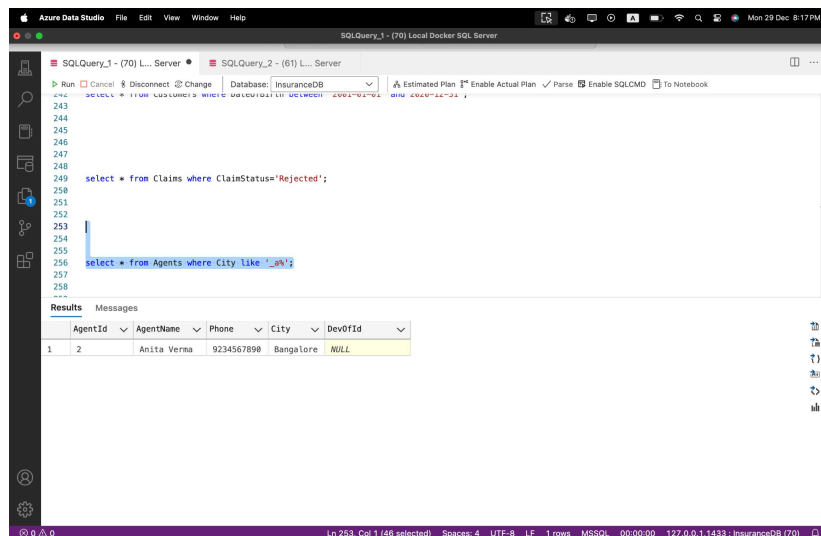
10. Display claims data where claim status is Rejected.

```
select * from Claims where ClaimStatus='Rejected';
```



11. Display records of Agents who stay in a city whose second letter is 'a';.

```
select * from Agents where City like '_a%';
```

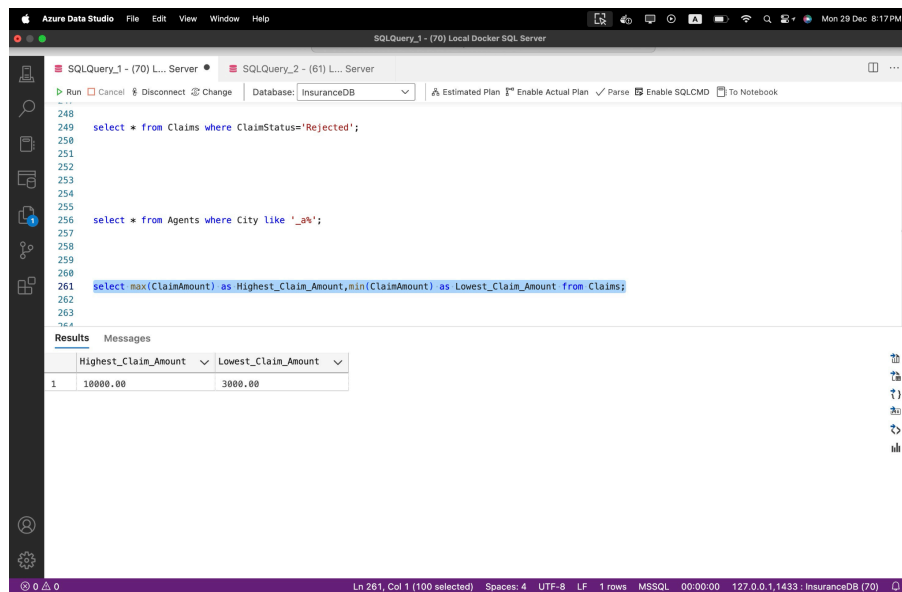


12. Display highest and lowest claimAmount from Claims table.

```
select max(ClaimAmount) as
```

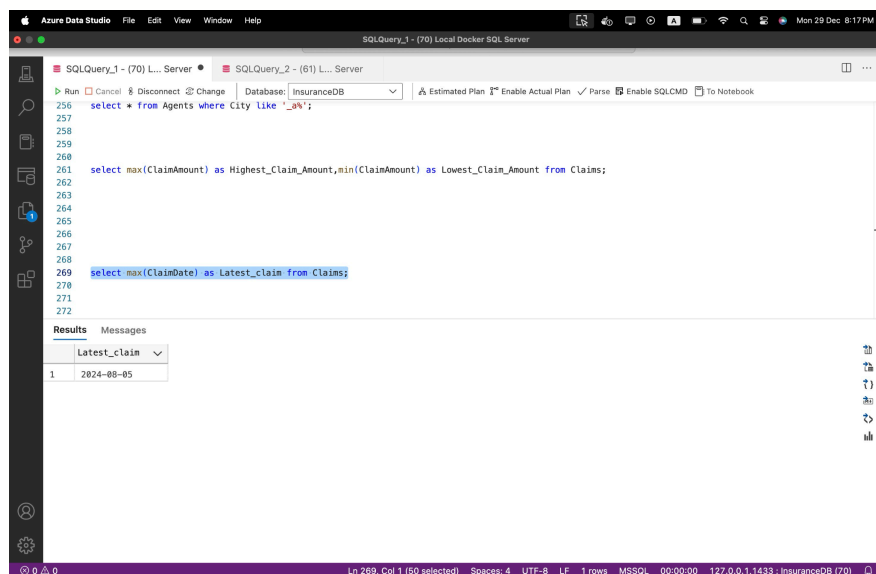
```
Highest_Claim_Amount,min(ClaimAmount) as
```

```
Lowest_Claim_Amount from Claims;
```



13. Display latest claim record.

```
select max(ClaimDate) as Latest_claim from Claims;
```



14. Increase premium amount to 10% for all health insurance policies.

```
select PremiumAmount*1.1 as Increased_Premium from
Policies where PolicyType='Health';
```

update Policies set PremiumAmount=1.1*PremiumAmount where PolicyType='Health';

The screenshot shows the Azure Data Studio interface with two SQL queries. The first query updates the PremiumAmount for Health policies. The second query selects the updated values.

```

263
264
265
266
267
268
269 select max(ClaimDate) as Latest_claim from Claims;
270
271
272
273
274 select * from Policies;
275 select PremiumAmount*1.1 as Increased_Premium from Policies where PolicyType='Health';
276 update Policies set PremiumAmount=1.1*PremiumAmount where PolicyType='Health';
277
278
279

```

Results

PolicyId	PolicyName	PolicyType	PremiumAmount	DurationYears
1	Health Secure	Health	12000.00	1
2	Term Life	Term Life Shield	25000.00	1
3	Motor Protect	Motor	8000.00	1

Increased_Premium

Increased_Premium
13200.00000

15. Delete the record of PolicyAssignments whose EndDate is before today's date.

select * from PolicyAssignments where EndDate<getdate();
delete from PolicyAssignments where EndDate<getdate();

The screenshot shows the Azure Data Studio interface with two SQL queries. The first query selects PolicyAssignments with an EndDate before today. The second query deletes these records.

```

271
272
273
274 select * from Policies;
275 select PremiumAmount*1.1 as Increased_Premium from Policies where PolicyType='Health';
276 update Policies set PremiumAmount=1.1*PremiumAmount where PolicyType='Health';
277
278
279
280
281
282 select * from PolicyAssignments where EndDate<getdate();
283 delete from PolicyAssignments where EndDate<getdate();
284
285
286
287

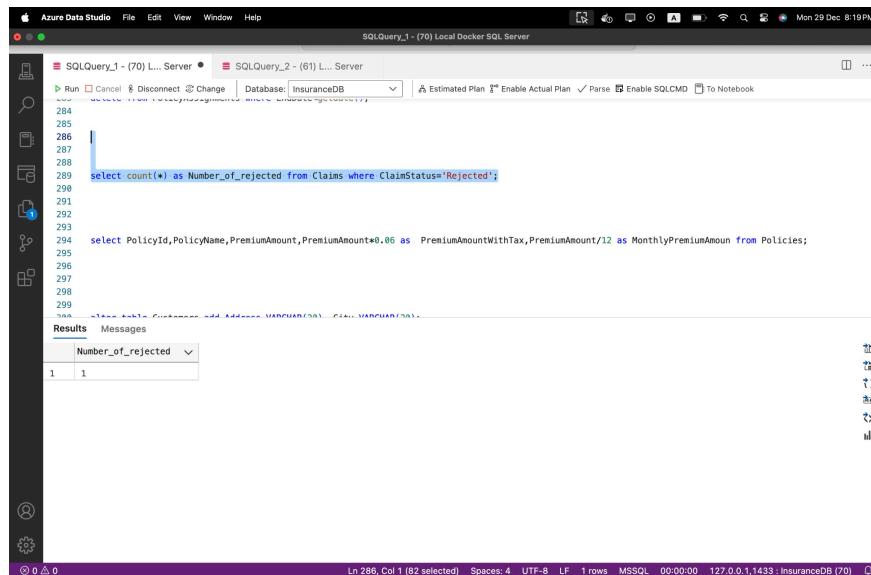
```

Results

AssignmentID	CustomerID	PolicyId	AgentId	StartDate	EndDate
1	3	1	1	2024-01-01	2024-12-31
2	4	3	2	2024-02-01	2025-01-31
3	5	3	2	2024-03-01	2025-02-28

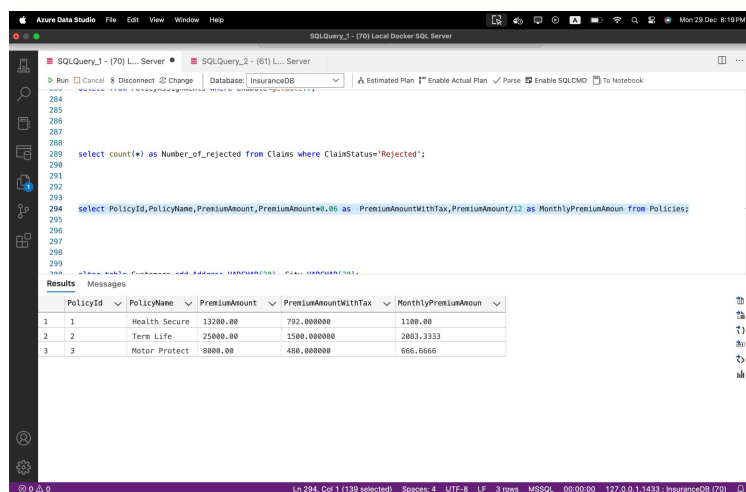
16. Display no of claims rejected.

```
select count(*) as Number_of_rejected from Claims where
ClaimStatus='Rejected';
```



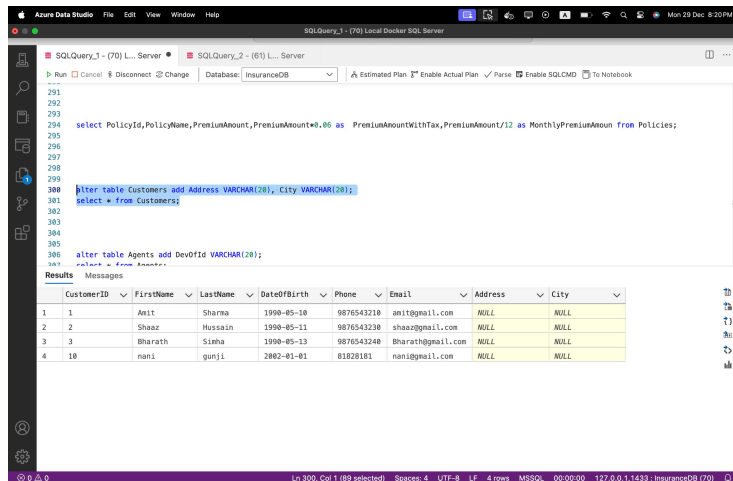
17. Display PolicyId, PolicyName, PremiumAmount along with computed fields not in table à 6% LocalTaxes, PremiumAmountWithTax and MonthlyPremiumAmount considering PremiumAmount is Annual.

```
select
PolicyId,PolicyName,PremiumAmount,PremiumAmount*0.06 as
PremiumAmountWithTax,PremiumAmount/12 as
MonthlyPremiumAmount from Policies;
```



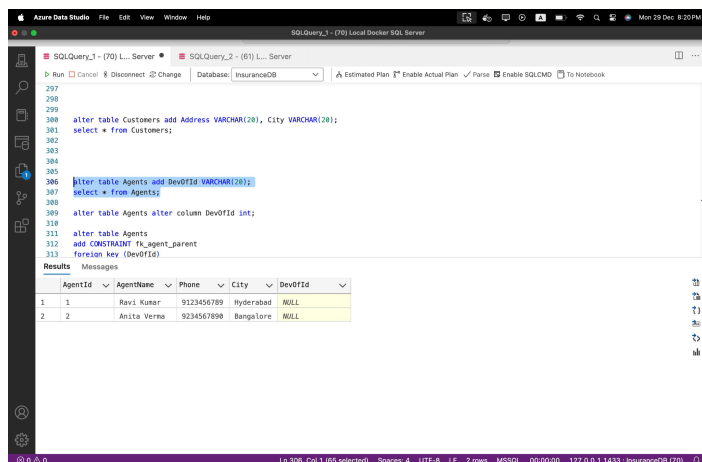
18. Write a command to add Address and City Columns in the Customers table.

```
alter table Customers add Address VARCHAR(20), City  
VARCHAR(20);  
select * from Customers;
```



19. Write a command to add a new column named DevOfId (DevelopmentOfficerId) in an existing Agents table.

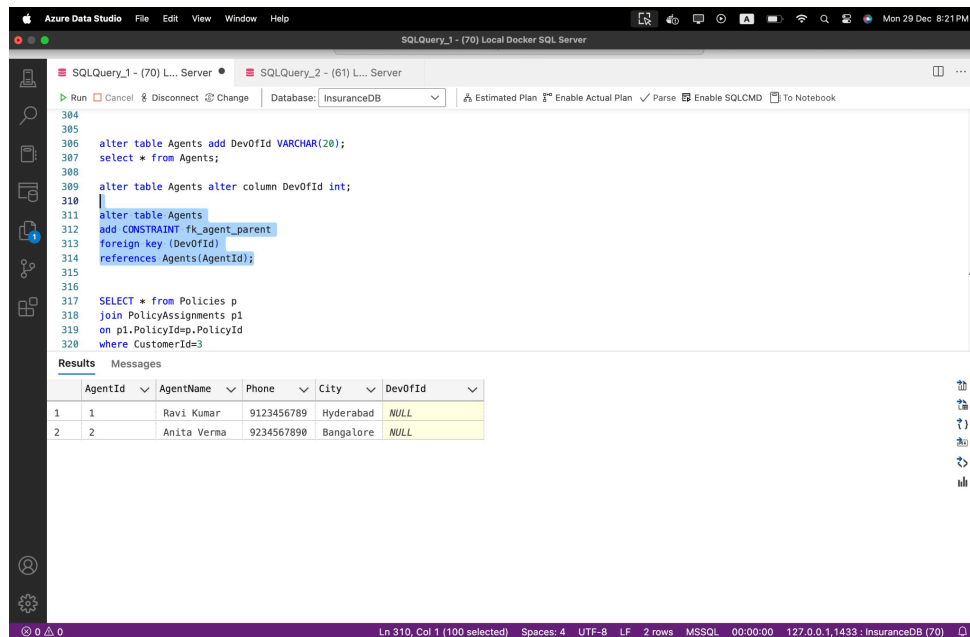
```
alter table Agents add DevOfId VARCHAR(20);  
select * from Agents;
```



20. Write command to make the above DevOfId as a recursive foreign key to AgentId as Parent.

```
alter table Agents  
add CONSTRAINT fk_agent_parent
```

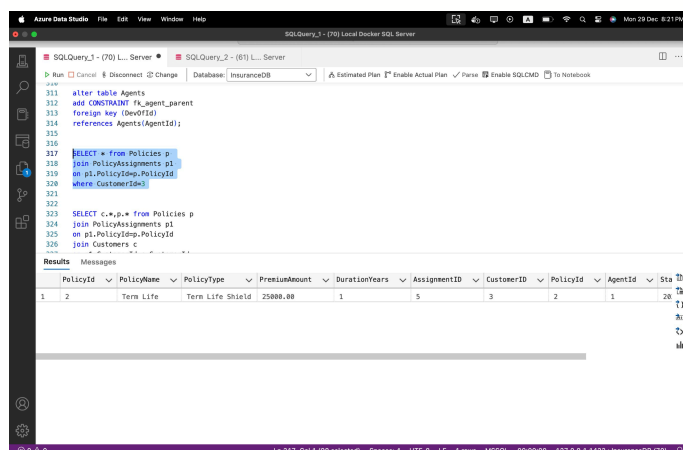
```
foreign key (DevOfId)
references Agents (AgentId);
```



5. Queries using Joins, Group By, Having etc.

1. List all Policies for a CustomerId 5.

```
SELECT * from Policies p
join PolicyAssignments p1
on p1.PolicyId=p.PolicyId
where CustomerId=3
```



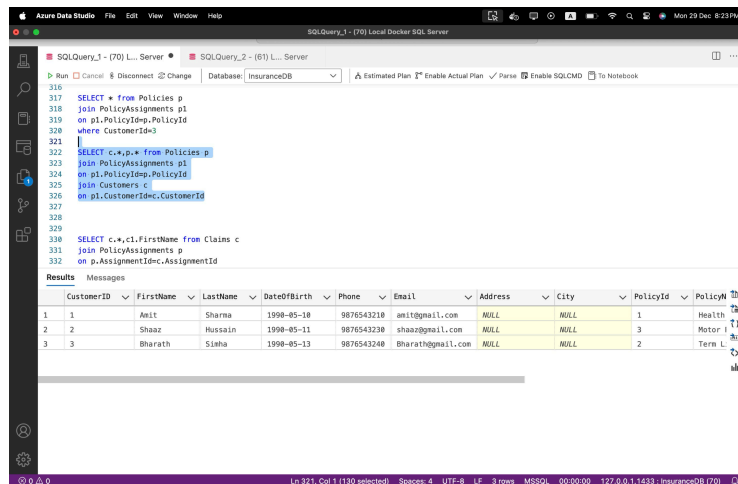
2. View all customers with their policies.

```
SELECT c.*,p.* from Policies p
join PolicyAssignments p1
```

```

on p1.PolicyId=p.PolicyId
join Customers c
on p1.CustomerId=c.CustomerId

```

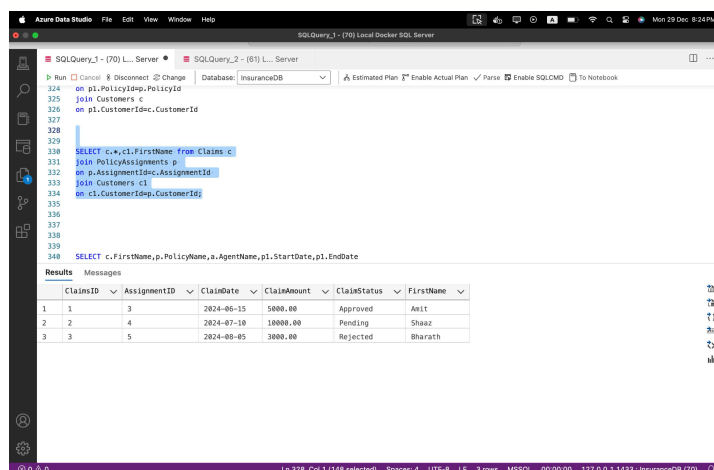


3. View claims with customer name.

```

SELECT c.*,c1.FirstName from Claims c
join PolicyAssignments p
on p.AssignmentId=c.AssignmentId
join Customers c1
on c1.CustomerId=p.CustomerId;

```



4. Display FirstName, PolicyName, AgentName, StartDate and EndDate from their respective tables.

SELECT

c.FirstName,p.PolicyName,a.AgentName,p1.StartDate,p1.EndDate

from PolicyAssignments p1

join Customers c

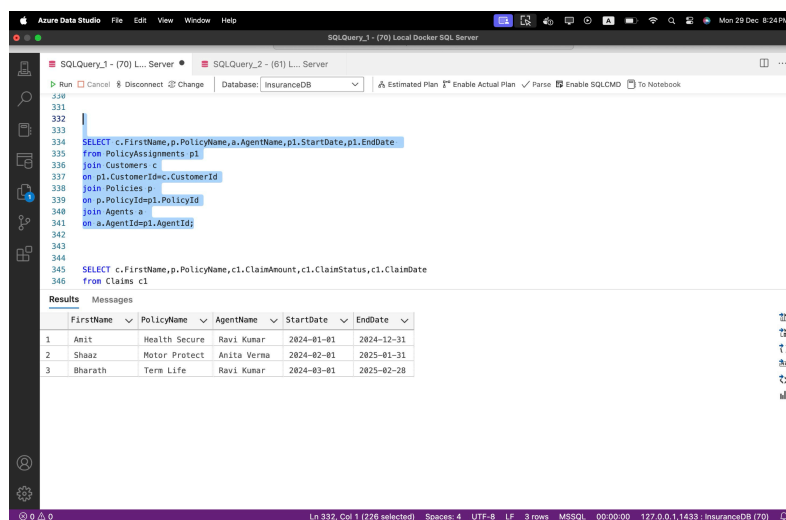
on p1.CustomerId=c.CustomerId

join Policies p

on p.PolicyId=p1.PolicyId

join Agents a

on a.AgentId=p1.AgentId;



5. Display claims report with FirstName, PolicyName, ClaimAmount, ClaimStatus, and ClaimDate from their respective tables.

SELECT

c.FirstName,p.PolicyName,c1.ClaimAmount,c1.ClaimStatus,c1.ClaimDate

from Claims c1

join PolicyAssignments p1

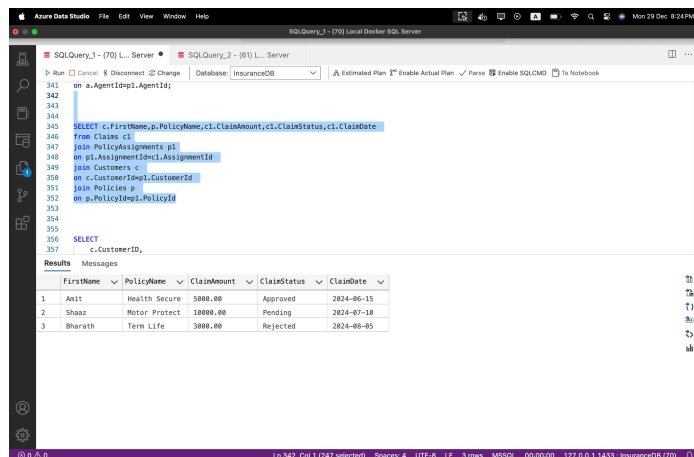
on p1.AssignmentId=c1.AssignmentId

join Customers c

on c.CustomerId=p1.CustomerId

join Policies p

on p.PolicyId=p1.PolicyId



6. Display records of Customers with or without Policies.

SELECT

c.CustomerID,
c.FirstName,
c.LastName,
p.PolicyName,
pa.StartDate,
pa.EndDate

FROM Customers c

LEFT JOIN PolicyAssignments pa

ON c.CustomerID = pa.CustomerID

LEFT JOIN Policies p

ON pa.PolicyId = p.PolicyId;

The screenshot shows the Azure Data Studio interface. The top menu bar includes File, Edit, View, Window, and Help. The main window displays a SQL query in the editor, with the database set to 'InsuranceDB'. The query is as follows:

```
354  
355  
356 SELECT  
357     c.CustomerID,  
358     c.FirstName,  
359     c.LastName,  
360     p.PolicyName,  
361     pa.StartDate,  
362     pa.EndDate  
363 FROM Customers c  
364 LEFT JOIN PolicyAssignments pa  
365     ON c.CustomerID = pa.CustomerID  
366 LEFT JOIN Policies p  
367     ON pa.PolicyId = p.PolicyId;  
368  
369  
370
```

Below the query editor, the 'Results' tab is active, showing a table with 7 columns: CustomerID, FirstName, LastName, PolicyName, StartDate, and EndDate. The table contains 4 rows of data:

	CustomerID	FirstName	LastName	PolicyName	StartDate	EndDate
1	1	Amit	Sharma	Health Secure	2024-01-01	2024-12-31
2	2	Shaaz	Hussain	Motor Protect	2024-02-01	2025-01-31
3	3	Bharath	Simha	Term Life	2024-03-01	2025-02-28
4	10	nani	gunji	NULL	NULL	NULL

The status bar at the bottom indicates the current position is Ln 354, Col 1 (248 selected), with 4 spaces, UTF-8 encoding, LF line endings, and 4 rows selected. The database is MSSQL, and the connection is 127.0.0.1,1433 : InsuranceDB (70).

7. Display all Customers with NO Claims.

```
SELECT DISTINCT  
    c.CustomerID,  
    c.FirstName,  
    c.LastName  
FROM Customers c  
LEFT JOIN PolicyAssignments pa  
    ON c.CustomerID = pa.CustomerID  
LEFT JOIN Claims cl  
    ON pa.AssignmentID = cl.AssignmentID
```

```
WHERE cl.ClaimsID IS NULL;
```

The screenshot shows the Azure Data Studio interface. The top bar indicates the current database is 'InsuranceDB'. The query editor contains the following SQL code:

```
370
371 SELECT DISTINCT
372     c.CustomerID,
373     c.FirstName,
374     c.LastName
375 FROM Customers c
376 LEFT JOIN PolicyAssignments pa
377     ON c.CustomerID = pa.CustomerID
378 LEFT JOIN Claims cl
379     ON pa.AssignmentID = cl.AssignmentID
380 WHERE cl.ClaimsID IS NULL;
381
382
383
384
385 SELECT
386     c.FirstName AS CustomerName
```

Below the query editor, the 'Results' tab is active, displaying a table with the following data:

	CustomerID	FirstName	LastName
1	10	nani	gunji

The status bar at the bottom shows the cursor is at line 371, column 1 (237 selected), with 4 spaces, UTF-8 encoding, LF line endings, 1 row, MSSQL engine, and a connection to 127.0.0.1,1433 : InsuranceDB (70).

8. Show CustomerName with Total Claim Amount per Customer.

```
SELECT
    c.FirstName AS CustomerName,
    SUM(cl.ClaimAmount) AS TotalClaimAmount
FROM Customers c
JOIN PolicyAssignments pa
    ON c.CustomerID = pa.CustomerID
JOIN Claims cl
    ON pa.AssignmentID = cl.AssignmentID
GROUP BY
    c.FirstName,
```

c.LastName;

The screenshot shows the Azure Data Studio interface. The top bar includes the application name and standard menu items. Below the menu bar, there are tabs for 'SQLQuery_1 - (70) Local Docker SQL Server' and 'SQLQuery_2 - (61) Local Docker SQL Server'. The main editor area displays an SQL query with line numbers 383 to 399. The query is a SELECT statement that joins 'Customers', 'PolicyAssignments', and 'Claims' tables, grouping by customer name and last name. Below the query editor, the 'Results' tab is active, showing a table with 3 columns: 'CustomerName', 'TotalClaimAmount', and an index. The table contains 3 rows of data. The status bar at the bottom shows the current cursor position (Ln 385, Col 1) and other details like 'Spaces: 4', 'UTF-8', 'LF', '3 rows', 'MSSQL', '00:00:00', and the connection string '127.0.0.1,1433 : InsuranceDB (70)'.

```
383
384
385 SELECT
386     c.FirstName AS CustomerName,
387     SUM(cl.ClaimAmount) AS TotalClaimAmount
388 FROM Customers c
389 JOIN PolicyAssignments pa
390     ON c.CustomerID = pa.CustomerID
391 JOIN Claims cl
392     ON pa.AssignmentID = cl.AssignmentID
393 GROUP BY
394     c.FirstName,
395     c.LastName;
```

	CustomerName	TotalClaimAmount
1	Shaaz	10000.00
2	Amit	5000.00
3	Bharath	3000.00

9. Show names and total claim amount of Customers With Claim Amount > 50000 (Use HAVING Clause).

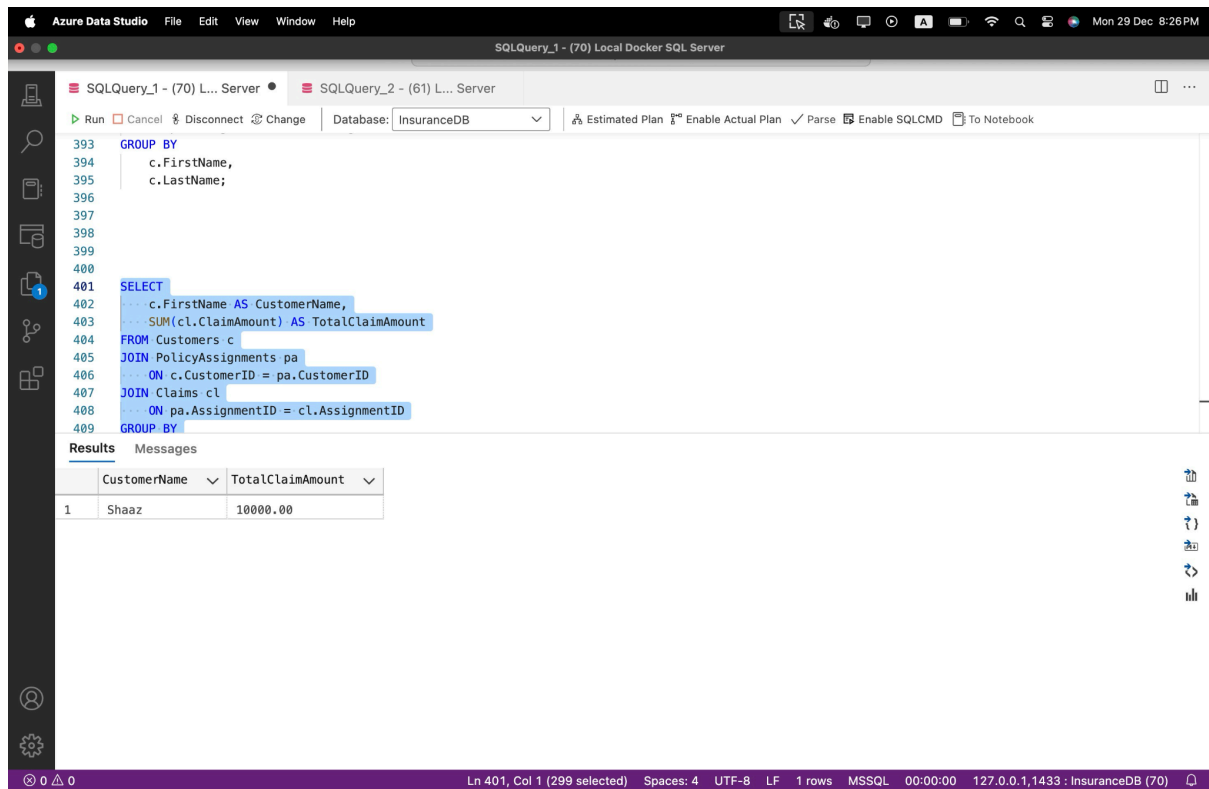
```
SELECT
    c.FirstName AS CustomerName,
    SUM(cl.ClaimAmount) AS TotalClaimAmount
FROM Customers c
JOIN PolicyAssignments pa
    ON c.CustomerID = pa.CustomerID
JOIN Claims cl
    ON pa.AssignmentID = cl.AssignmentID
GROUP BY
```



```
c.FirstName,  
c.LastName
```

HAVING

```
SUM(cl.ClaimAmount) > 5000;
```



10. Display list with Agent Wise Policy Count.

```
SELECT  
    a.AgentName,  
    COUNT(pa.PolicyId) AS PolicyCount  
FROM Agents a  
LEFT JOIN PolicyAssignments pa  
    ON a.AgentId = pa.AgentId  
GROUP BY  
    a.AgentName;
```

SQLQuery_1 - (70) Local Docker SQL Server

SQLQuery_1 - (70) L... ServerSQLQuery_2 - (61) L... Server

RunCancelDisconnectChangeDatabase: InsuranceDBEstimated PlanEnable Actual PlanParseEnable SQLCMDTo Notebook

413

414

415

416

417

418

419

420

421

422

423

424

425

426

427

428

429

SELECT

... a.AgentName,

... COUNT(pa.PolicyId) AS PolicyCount

FROM Agents a

LEFT JOIN PolicyAssignments pa

ON a.AgentId = pa.AgentId

GROUP BY

... a.AgentName;

ResultsMessages

	AgentName	PolicyCount
1	Anita Verma	1
2	Ravi Kumar	2

Ln 417, Col 1 (163 selected)Spaces: 4UTF-8LF2 rowsMSSQL00:00:00127.0.0.1,1433 : InsuranceDB (70)

	AgentName	PolicyCount
1	Anita Verma	1
2	Ravi Kumar	2